

1 - NuSMV

NuSMV is a symbolic model checker developed by ITC-IRST and UniTN with the collaboration of CMU and UniGE. The NuSMV project aims at the development of a state-of-the-art model checker that:

- Is robust, open and customizable;
- Can be applied in technology transfer projects;
- Can be used as research tool in different domains.

NuSMV is OpenSource developed by a distributed community, “Free Software” license.

NuSMV provides:

1. A language for describing finite state models of systems
 - Reasonably expressive
 - Allows for modular construction of models
2. Model checking algorithms for checking specifications written in LTL and CTL (and some other logics) against finite state machines.

In this exercise you verify properties of a model of a FIFO digital circuit by using NuSMV. Also, you can find User Manual and Tutorial of NuSMV in the folder.

2 - Description of provided FIFO model

A block diagram of the FIFO circuit is shown in Figure 1.

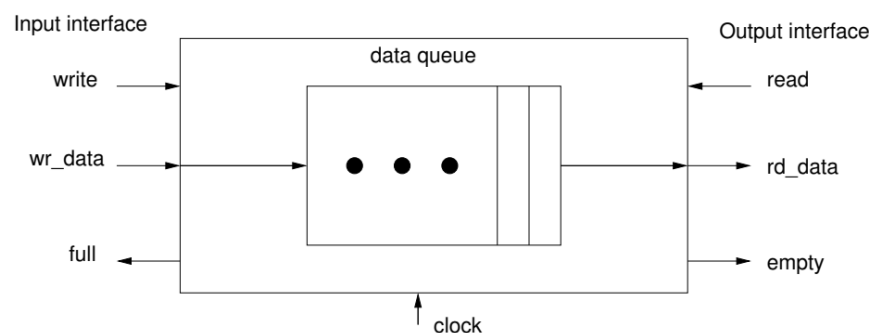


Figure 1. FIFO block diagram

Abstractly, a FIFO is a variable-length queue of data words. It has two interfaces, one *input interface* for adding words to one end of the queue and one *output interface* for reading and removing words from the other end of the queue.

The hardware circuit is a *synchronous* circuit. Its behaviour is governed by a Boolean clock signal input which usually alternates between *true* and *false* at a uniform frequency.

Each time the clock changes from *false* to *true*, the internal state of the circuit is updated based on the current internal state and inputs to the circuit at that time. When modelling a synchronous circuit in NuSMV, we do not explicitly include the clock signal. Rather, we design a transition system that takes one step per clock cycle and that uses the transition relation to specify how the internal state is updated based on the current state and inputs. In general synchronous circuits might have outputs that depend both on the current state and the inputs. Here we use a restriction of this scheme where the outputs depend only on the current state. For simplicity, the following description of FIFO behaviour is in terms of the transition system model rather than the hardware circuit.

To add or write a word of data to the FIFO, the data is presented on the *write data* `wr_data` input and the Boolean signal *write* is asserted (set to *true*). Providing the FIFO is not currently full, the write data word is then added to the queue on the transition to the next step. The FIFO has a maximum number of words it can hold in the queue at any one time. The Boolean *full* output of the FIFO indicates whether or not it currently holds the maximum number.

The *read data* `rd_data` output of the FIFO shows the current end word in the FIFO's internal queue, providing that the queue is not empty. The queue being empty is signalled by the Boolean *empty* output being set to *true*. If the Boolean *read* signal is set to *true* and the queue is not empty, on the transition to the next step the current end word in the queue is removed and the word behind it (if any) then appears on the FIFO `rd_data` output.

The provided file `fifo.smv` presents the NuSMV FIFO model. Have a look at the model. For simplicity and to ensure rapid NuSMV execution times, we set the `DEPTH` constant for the maximum number of words to 5 and the `WIDTH` constant for the word size to 1. In practice we would often use larger values for both parameters.

Internally, the FIFO uses a *circular buffer* to implement the queue. This consists of an array buffer of words of size `DEPTH` and two pointers into this array, the *read pointer* `rd_p` and the *write pointer* `wr_p`. If the queue is not empty, the read pointer points to word which is the current output word of the queue and, if the queue is not full, the write pointer points to the position to write the next input word. When a new word is written into the queue, the write pointer is incremented, wrapping it around as necessary. When a word in the queue is removed, the read pointer is incremented, wrapping it around as necessary. See Figure 2 for two examples of the internal configuration of the FIFO when the queue holds the words `w0`, `w1` and `w2`, added in that order.

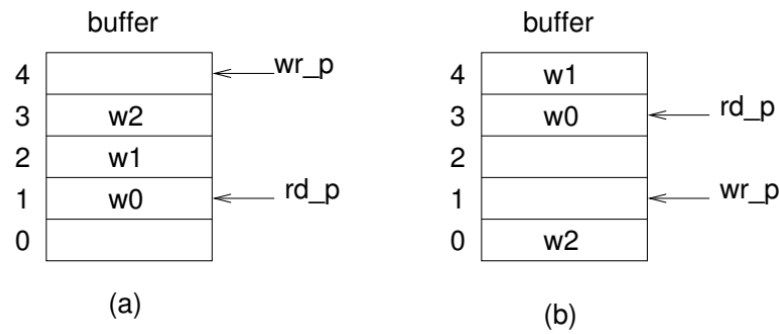


Figure 2. Examples of internal FIFO configurations

With the provided FIFO realisation, the FIFO could be either empty or full when the two pointers are equal. The design uses the Boolean empty internal state variable to distinguish between these two cases.

3 - Properties to verify

In the provided template file `fifo-properties.smv`, add formulas for the LTL and CTL properties requested below. Verify your properties with NuSMV by running the command:

`NuSMV -pre cpp fifo.smv`

The `fifo.smv` file brings in the `fifo-properties.smv` file using a preprocessor `#include` directive at its end. The `-pre cpp` option to NuSMV here is necessary to ensure it runs the C preprocessor on `fifo.smv` in order to interpret this directive. You will get warnings until you fill in the LTL specifications. If you don't want to see counter-examples for false formulas, also add the `-dcx` option.

1. Write LTL formulas for the following properties (all true of the FIFO model in `fifo.smv`):

- (a) *It is never the case that the FIFO indicates simultaneously it is both empty and full.*
- (b) *If write is asserted forever and read is never asserted, then the FIFO eventually becomes full.*
- (c) *At any time, if a 1 is presented to the FIFO data input and write is asserted, then eventually a 1 will appear on the FIFO data output.*

With further reasonable assumptions added after the *if* concerning FIFO signals such as read, empty or full, to ensure the property checks true. Consult the NuSMV user guide for information on *word constants* to be able to refer to the number 1.

- (d) The same property as in (c), except that it is phrased to hold for any data value, not just the value 1. Use the 'frozen variable' `data1` to do this. Consult the NuSMV user guide for documentation on *frozen variables* (also sometimes known in temporal logic as *rigid variables*).

(e) the same property as in (d), except that, in addition, it requires the empty output of the FIFO to be set to false at all times *inbetween* the time the write of the data is set up and the time the data can first be read out, but not actually at either of these times. You may take advantage of the fact that the earliest we expect the data to appear is the step after it is written.

(f) a similar property to that for (d), except that it assumes that two possibly-distinct data values are input on consecutive steps, and checks for the same two values appearing on the output on consecutive steps. Use the provided frozen variables `data1` and `data2` to refer to the two data values.

2. Write CTL formulas for

(a) *There exists a run for which, at some time onwards, the FIFO is always full,*

(b) *From every reachable state in which the FIFO is full, there exists a path along which the FIFO eventually becomes empty.*

Both of the above properties should be found true of the FIFO model in `fifo.smv`.